A aprendizagem de Máquina - II

Atividade para Compensação de Aula (19.09.2026)
Prof. Luis Claudio

ATENÇÃO:

Após a data limite, haverá desconto de uma falta por dia de atraso.

Aluno: Lucas Gomes Silva

Situação-Problema 01 — Configuração e treinamento de uma MLP

Uma empresa deseja utilizar uma rede neural para classificar clientes entre duas categorias. Após a preparação dos dados, cada cliente é representado por **20 atributos numéricos**. A equipe construiu inicialmente uma MLP com duas camadas ocultas, contendo **64 e 32 neurônios**, ambas utilizando ReLU, e uma camada de saída com **1 neurônio e Sigmoid**.

O treinamento foi configurado com batch_size = 128, epochs = 40, learning_rate = 0,001 e otimizador Adam. Durante os testes, a equipe percebeu que diferentes configurações produzem resultados distintos e decidiu avaliar alterações tanto na arquitetura quanto no processo de treinamento.

Questões

1) Identifique, entre as configurações apresentadas, quais correspondem a **hiperparâmetros de arquitetura** e quais correspondem a **hiperparâmetros de treinamento**. Explique também por que pesos e biases não pertencem a nenhuma dessas duas categorias.

Quantidade de camadas ocultas, número de neurônios, funçõe de ativação e quantidade de neurônios são hiperparâmetros de arquitetura pois definem a estrutura da rede neural.

batch_size, epochs, learning_rate e otimizador Adam são hiperparâmetros de treinamento pois controlam como a rede aprende.

Pesos e biases não são hiperparâmetros pois eles não são definidos previamente, eles são aprendimentos automaticamente durante o treinamento e atualizados pelo algoritmo de otimização.

2) A equipe pretende testar diferentes quantidades de camadas ocultas, neurônios e funções de ativação. Explique por que essas escolhas precisam ser avaliadas experimentalmente e como elas podem afetar a capacidade de representação da rede.

Essas escolhas precisam ser testadas porque não existe uma configuração perfeita que seja a melhor para todos os problemas. A

Aumentar o número de camadas e neurônicos aumenta a capacidade de representação da rede, mas pode aumentar muito o custo computacional e favorer overfitting, já uma rede pequena demais por causar underfitting.

Funções de ativação podem ser usadas na camada de oculta e na camada de saída com objetivos diferentes, uma vez que cada função de ativação serve para um tipo de problema.

3) Considere agora alterações no batch_size, no número de épocas e no learning_rate. Explique como cada uma dessas alterações pode modificar o processo de treinamento, incluindo quantidade de atualizações, estabilidade do gradiente, velocidade de aprendizado e possibilidade de underfitting ou overfitting.

O batch_size indica quantas amostras são utlizadas antes de uma atualização de pesos. Um bach menor gera mais atualizações por época, podendo ajudar na generalização, mas também pode tornar o treinamento menos estável. Um batch maior produz gradiantes mais estáveis e permite maior paralelização, porém gera menos atualizações por época.

O número de épocas determina quantas vezes todo o conjunto de treinamento é apresentado a rede. Poucas épocas podem gerar underfitting, porque a rede não teve tempo suficnete para aprender. Muitas épocas podem levar ao overfitting.

O learning rate determina o tamanho das alterações feitas nos pesos a cada atualização. Um rate muito alto pode fazer o treinamento oscilar. Um valor muito baixo tende a produzir um aprendizado mais estável, porém muito mais lento.

4) Explique, de forma integrada, o papel da **função de perda, do gradiente, do learning rate e do otimizador** durante o treinamento. Deixe claro por que o gradiente não é um hiperparâmetro.

A função de perda mede o quanto as previsão da rede estão diferente dos valores esperados. A partir dessa perda é calculado os gradiantes a través do processo de backpropagation, para indicar como cada peso influencia o valor da perda e em qual direção ele deve ser modificado para tentar reduzí-la.

O learning rate controla o tamanho do passo dado na direção indicada pelo gradiente.

O otimizador utiliza esses gradiantes para determinar como os pesos e biases serão efetivamente atualizados.

O gradiante não é um hiperparâmetro porque ele é calculado automaticamente a cada etapa, dependendo dos dados, dos pesos e da função da perda. Ele não é escolhido antes do treinamento.

Situação-Problema 02 — Classificação de imagens utilizando CNN

Uma indústria deseja classificar imagens coloridas de produtos para identificar possíveis defeitos. Cada imagem possui dimensão **32 × 32 × 3**. A primeira camada convolucional utiliza **16 filtros 3 × 3**, stride 1, padding same e função ReLU. A rede possui posteriormente outra camada convolucional, uma operação de Max Pooling e, antes das camadas densas, uma operação Flatten.

Durante o desenvolvimento, alguns integrantes da equipe confundiram conceitos como canais, filtros, mapas de características e quantidade de parâmetros, além de questionarem por que utilizar uma CNN em vez de simplesmente transformar toda a imagem em um vetor e utilizar uma MLP.

Questões

5) Explique como um filtro 3 × 3 atua sobre uma imagem RGB. Indique a dimensão completa de cada filtro, como os três canais participam do cálculo e quantos mapas de características são produzidos pelos 16 filtros.

Como a imagem possui três canais (RGB), a dimensão completa é 3x3x3. Isso significa que cada filtro possui uma matriz de 3x3 para cada um dos canais.

Em cada posição da imagem, os valores da matriz são multiplicados pelos pesos correspondentes do filtro, depois os resultados soão somados e é adicionado um bias, assim cada filtro produz um mapa de características.

Como existem 16 filtros, serão produzidos 16 maps de características.

6) Calcule a quantidade de parâmetros treináveis dessa primeira camada convolucional. Em seguida, explique por que essa quantidade não aumenta quando a altura e a largura da imagem aumentam, desde que sejam mantidos a quantidade e o tamanho dos filtros e a quantidade de canais.

Cada filtro possui 3x3x3: 27 pesos.

Cada filtro possui 1 bias: 27+1 = 28

*Como existem 16 filtros: 28*16 = 448 parâmetros treináveis.*

A quantidade de parâmetros não depende da altura ou largura da imagem poque os mesmos pesos do filtro são reutilizados em diferentes posições da imagem. Uma imagem maior gera mais posições onde o filtro será aplicado, mas não exige novos pesos.

7) Explique como **stride e padding** influenciam as dimensões espaciais dos mapas de características. Em sua resposta, diferencie same e valid.

O stride representa o tamanho do deslocamento do filtro sobre a imagem. Por exemplo, se stride = 1 o filtro avança uma posição por vez, com stride = 2, ele avança duas posições, reduzindo as dimensões espaciais do mapa produzido.

O padding adiciona pixels extras ao redor das bordas de uma imagem ou matriz de entrada antes de aplicar uma operação de convolução.

No padding valid, nenhum preenchimento é adicionado ao redor da imagem.

No padding same, são adicionadas bordas, normalmente com zeros, para preservar as dimensões espaciais.

8) A primeira camada produz 16 mapas de características, que são recebidos pela segunda camada convolucional. Explique qual será a **profundidade de cada filtro da segunda camada**, quantos mapas de características serão produzidos pelos seus 32 filtros e qual é a função do **Flatten** antes das camadas densas.

A primeira camada produz uma saída de dimensão 32 × 32 × 16, portanto a entrada da segunda camada convolucional possui 16 canais. Assim, a profundidade de cada filtro da segunda camada deve ser 16, independentemente do tamanho espacial do filtro.

Como a segunda camada possui 32 filtros, e cada filtro produz um mapa de características, serão gerados 32 mapas de características. Portanto, a saída dessa camada terá profundidade 32.

Depois das convoluções e da operação de Max Pooling, utiliza-se o Flatten, que tem como função transformar os mapas de características multidimensionais em um vetor unidimensional, permitindo que esses dados sejam utilizados pelas camadas densas da rede.